

Manual Operativo de Despliegue y Administración

Guía de producción para servidores locales, virtualizados y entornos on-premises

Código del Documento:	CREAPP-OPS-01
Versión:	2.0 (Edición On-Premises)
Autor:	Juan Martín Morales
Repositorio:	https://github.com/juanm4morales/cre-app
Clasificación:	Documentación Técnica de Operaciones
Destinatarios:	Administradores de Sistemas, Infraestructura y DevOps
Fecha de Revisión:	30 de septiembre de 2026

Contenido

Propósito y alcance	1
Guía de instalación y operación para servidores locales	1
1. Arquitectura y Requisitos Mínimos	1
2. Método recomendado: PostgreSQL nativo + Gunicorn + Nginx	2
4. RespalDOS y Restauración	6
5. Despliegue en Cloud Provider (Microsoft Azure)	6
6. Solución Rápida de Problemas Comunes	8
7. Documentación Oficial y Fuentes de Referencia	9

Propósito y alcance

Guía de instalación y operación para servidores locales

Autor: Juan Martín Morales

Repositorio: <https://github.com/juanm4morales/cre-app>

Versión: 2.0 (Edición On-Premises)

Este manual describe el procedimiento para desplegar **CREApp** en un servidor local o máquina virtual (on-premises), así como las pautas de publicación en proveedores cloud (Microsoft Azure), de forma rápida, robusta y directa.

1. Arquitectura y Requisitos Mínimos

CREApp está compuesta por:

- **Frontend:** Aplicación web ([React](#) / TypeScript compilada con [Vite](#)). Nginx la entrega directamente como archivos estáticos.
- **Backend:** API REST en [Django](#) (Python) servida mediante [Gunicorn WSGI](#).
- **Base de Datos:** [PostgreSQL](#) para persistencia relacional con transacciones ACID.
- **Proxy Inverso:** [Nginx](#) para terminación TLS/HTTPS y derivación de peticiones.

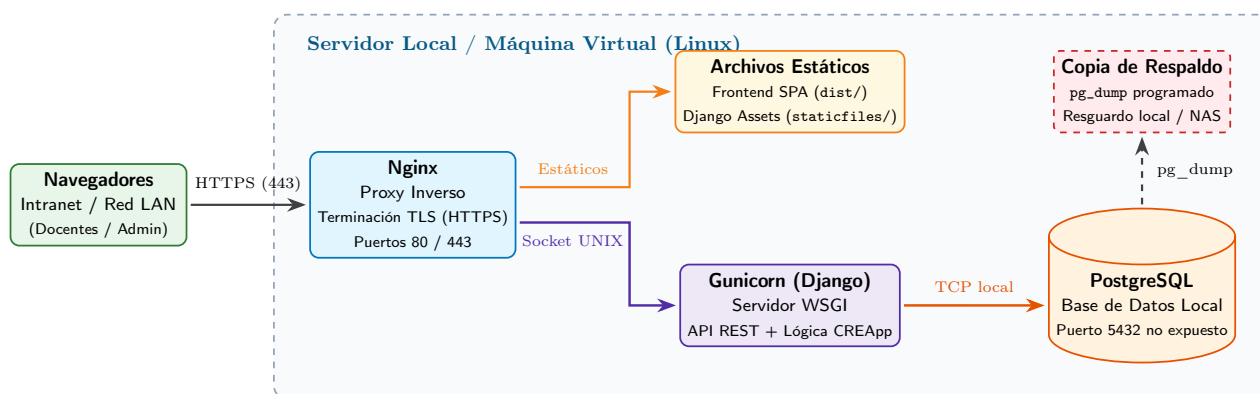


Figura 1. Topología de arquitectura en servidor local

Requisitos mínimos del sistema

- **Hardware:** 1 vCPU, 2 GB de memoria RAM y 10 GB de disco.
- **Sistema Operativo:** Ejemplos de usuarios, grupos y rutas para Debian/Ubuntu; adapte el usuario del servidor web, servicios y rutas en otras distribuciones.
- **Software base:**
 - Python 3.13 con soporte para [entornos virtuales \(venv\)](#).
 - PostgreSQL 17 (versión utilizada en el Compose de desarrollo).
 - Node.js 22 y npm (necesario para compilar el frontend).
 - Nginx (o cualquier proxy inverso equivalente).

Ramas del repositorio y selección para despliegue

- **azure (Base de referencia):** Rama same-origin (VITE_API_URL=/api, VITE_STATIC_BASE=/api/static/, prefijo de admin /api/sadmin-creapp-panel/). Para un servidor propio, despliegue un SHA revisado y aprobado derivado de esta rama; no despliegue main por defecto.

- **azure-same-origin:** Variante técnica derivada de **azure**. La divergencia intencional respecto a **azure** se concentra exclusivamente en archivos de enrutamiento y despliegue (**backend/config/settings.py**, **backend/config/urls.py**, **frontend/src/App.tsx**, **frontend/vite.config.js**). Los cambios de producto o dominio deben incorporarse primero en **azure** y propagarse hacia adelante (*sync forward*).
 - **main:** Rama histórica (*trunk*). **Advertencia:** En GitHub Actions tiene asociados dos flujos de trabajo concurrentes (**deploy.yml** y **main_cre-app-api.yml**). Se recomienda utilizar **azure** como rama oficial de despliegue.
 - **dev:** Rama de integración continua y desarrollo de nuevas características.
 - **docs/developer-handbook:** Rama de documentación técnica, manuales operativos (LaTeX/Markdown) y diagramas de arquitectura.
-

2. Método recomendado: PostgreSQL nativo + Gunicorn + Nginx

Para administradores que prefieren gestionar los servicios directamente en el sistema operativo mediante [systemd](#):

Paso 1: Aprobar el código y preparar el checkout

No use el **main** predeterminado ni una rama móvil como release. Confirme el SHA completo revisado y aprobado de **azure** (o de una rama/release derivada de **azure** que incluya ese cambio), registre ese SHA y haga checkout detached. Desplegar en este servidor no requiere ni debe implicar hacer push a **azure**.

```
sudo useradd --system --no-create-home --home-dir /srv/creapp --shell /usr/sbin/nologin --gid
↳ www-data creapp
sudo install -d -o creapp -g www-data -m 0750 /srv/creapp
sudo -u creapp git clone --branch azure --no-checkout
↳ https://github.com/juanm4morales/cre-app.git /srv/creapp
sudo -u creapp git -C /srv/creapp fetch origin azure
sudo -u creapp git -C /srv/creapp checkout --detach <SHA_COMPLETO_REVISADO>
sudo -u creapp git -C /srv/creapp rev-parse HEAD
```

Compare **git rev-parse HEAD** con el SHA aprobado y deténgase si no coincide. En una instalación real, el checkout y código deben pertenecer al operador de despliegue (**creapp** aquí), no a **www-data** ni al proceso web.

Paso 2: Crear la base de datos en PostgreSQL

Conéctese a PostgreSQL y cree el usuario y la base conforme a las directivas de seguridad de [roles y privilegios de PostgreSQL](#):

```
CREATE USER creapp_usr WITH PASSWORD 'contraseña_segura';
CREATE DATABASE creapp_db OWNER creapp_usr ENCODING 'UTF8';
```

Paso 3: Crear el archivo de entorno

Guarde **/etc/creapp/creapp.env** como asignaciones **NOMBRE=valor** compatibles con **EnvironmentFile** de **systemd**, propiedad **root:creapp** y modo **0640** (no haga **source** de este archivo en una shell). Proteja el secreto y contraseña; no los copie al checkout. Configure valores reales y flags de HTTPS:

```
DEBUG=False
SECRET_KEY=generar_una_clave_aleatoria_segura_de_50_caracteres
ALLOWED_HOSTS=creapp.institucion.edu.ar,192.168.1.50,localhost
POSTGRES_DB=creapp_db
POSTGRES_USER=creapp_usr
POSTGRES_PASSWORD=contraseña_segura
```

```

POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
CSRF_TRUSTED_ORIGINS=https://creapp.institucion.edu.ar
CORS_ALLOWED_ORIGINS=https://creapp.institucion.edu.ar
CSRF_COOKIE_SECURE=True
SESSION_COOKIE_SECURE=True

```

(Consulte la documentación de [ALLOWED_HOSTS](#), [CSRF_TRUSTED_ORIGINS](#) y cookies de seguridad en Django).

Paso 4: Instalar, construir y desplegar

Use directorios de trabajo explícitos. Antes de Django `collectstatic`, compile la SPA: los archivos generados son parte de `STATICFILES_DIRS`. Las migraciones se ejecutan solo en la DB aprobada, después de revisar el plan y tomar backup; no se ejecutan automáticamente al iniciar el servicio. En una instalación nueva, confirme que dispone de los datos iniciales necesarios y de un seed/importador corregido y probado para una base vacía; `migrate` no garantiza todos los datos operativos.

```

# Desde /srv/creapp, como operador de despliegue
python3 -m venv /srv/creapp/.venv
/srv/creapp/.venv/bin/pip install --upgrade pip
/srv/creapp/.venv/bin/pip install -r /srv/creapp/requirements.txt

# npm ci y build son previos a collectstatic; Vite incorpora estas variables al build.
cd /srv/creapp/frontend
npm ci
VITE_API_URL=/api VITE_STATIC_BASE=/api/static/ npm run build

# Cargar el entorno de forma segura mediante systemd (no source del archivo).
# Revisar el plan antes de aplicar cambios a la DB autorizada.
sudo systemd-run --wait --pipe --collect \
  -p User=creapp -p Group=www-data \
  -p WorkingDirectory=/srv/creapp/backend \
  -p EnvironmentFile=/etc/creapp/creapp.env \
  /srv/creapp/.venv/bin/python manage.py migrate --plan

# Ejecutar solo con autorización, backup confirmado y ventana aprobada.
sudo systemd-run --wait --pipe --collect \
  -p User=creapp -p Group=www-data \
  -p WorkingDirectory=/srv/creapp/backend \
  -p EnvironmentFile=/etc/creapp/creapp.env \
  /srv/creapp/.venv/bin/python manage.py migrate --noinput

# creapp escribe STATIC_ROOT; Nginx debe poder leerlo.
sudo install -d -o creapp -g www-data -m 0750 /srv/creapp/backend/staticfiles
sudo systemd-run --wait --pipe --collect \
  -p User=creapp -p Group=www-data \
  -p WorkingDirectory=/srv/creapp/backend \
  -p EnvironmentFile=/etc/creapp/creapp.env \
  /srv/creapp/.venv/bin/python manage.py collectstatic --noinput

```

(Los comandos administrativos corresponden a `manage.py migrate`, `collectstatic` y al empaquetado de producción de [Vite Build](#)).

Después de aplicar migraciones, cree deliberadamente una cuenta administrativa si corresponde. Ejecute `createsuperuser` interactivamente; no ponga contraseñas en argumentos, logs ni archivos de entorno:

```

sudo systemd-run --pty --wait --collect \

```

```
-p User=creapp -p Group=www-data \
-p WorkingDirectory=/srv/creapp/backend \
-p EnvironmentFile=/etc/creapp/creapp.env \
/srv/creapp/.venv/bin/python manage.py createsuperuser
```

Paso 5: Configurar el servicio Systemd (/etc/systemd/system/creapp.service)

Cree el servicio para que Gunicorn arranque automáticamente, siguiendo las directivas de [integración de Gunicorn con Systemd](#):

```
[Unit]
Description=CREApp Django Service
After=network.target postgresql.service

[Service]
Type=simple
User=creapp
Group=www-data
WorkingDirectory=/srv/creapp/backend
EnvironmentFile=/etc/creapp/creapp.env
ExecStart=/srv/creapp/.venv/bin/gunicorn \
    --workers 3 \
    --bind 127.0.0.1:8000 \
    config.wsgi:application
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Active e inicie el servicio:

```
sudo systemctl daemon-reload
sudo systemctl enable --now creapp
```

El código y `frontend/dist` deben ser legibles y sus directorios padre atravesables por el proceso. En este ejemplo `creapp` usa grupo `www-data`, que permite a Nginx leer el checkout; el usuario del servicio necesita escribir en `backend/staticfiles` para `collectstatic`. Revise propietario, grupo y modos efectivos después de clonar/build.

Paso 6: Configurar Nginx (/etc/nginx/sites-available/creapp)

Cree la configuración del proxy inverso basada en el módulo [ngx_http_proxy_module](#) y las recomendaciones para [servidores HTTPS seguros](#):

```
server {
    listen 80;
    server_name creapp.institucion.edu.ar;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name creapp.institucion.edu.ar;

    # Reemplace por las rutas a un certificado real y válido para este FQDN.
    ssl_certificate      /etc/letsencrypt/live/creapp.institucion.edu.ar/fullchain.pem;
```

```

ssl_certificate_key /etc/letsencrypt/live/creapp.institucion.edu.ar/privkey.pem;

client_max_body_size 25M;

# 1. Archivos estáticos de Django
location ~^ /api/static/ {
    alias /srv/creapp/backend/staticfiles/;
    expires 30d;
    access_log off;
}

# 2. Rutas dinámicas hacia Django (Gunicorn)
location /api/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location = /healthz {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# 3. Interfaz de usuario (React SPA)
location / {
    root /srv/creapp/frontend/dist;
    index index.html;
    try_files $uri $uri/ /index.html;
}
}

```

El certificado real debe existir antes de cargar esta configuración. Gunicorn escucha únicamente en loopback; no exponga 8000. No configure PROXY_ACCESS_SECRET ni envíe X-Proxy-Access-Secret salvo que un proxy lo requiera y esté configurado intencionalmente.

Habilite el sitio y valide su sintaxis antes de recargar Nginx:

```

sudo ln -sf /etc/nginx/sites-available/creapp /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

(Para la obtención y renovación automática de certificados TLS gratuitos, se sugiere utilizar [Certbot](#) / [EFF](#)).

Paso 7: Pruebas smoke

Con DNS y TLS reales listos, no se conforme solo con un status 200. Compruebe cuerpo y Content-Type; el ejemplo JSON usa una ruta existente (/api/auth/csrf) que no requiere sesión:

```

curl -fsS -D - https://creapp.institucion.edu.ar/healthz
# Debe incluir Content-Type: text/plain y cuerpo `ok`.
curl -fsS -D - https://creapp.institucion.edu.ar/api/auth/csrf
# Debe devolver JSON (Content-Type application/json), no el index.html de la SPA.
# Sustituya por una ruta real de JS/CSS listada en frontend/dist/index.html.

```

```
curl -fsS -D - 'https://creapp.institucion.edu.ar/api/static/assets/PEGAR_NOMBRE_REAL.js'  
# Debe devolver JavaScript/CSS con su Content-Type correcto.  
curl -fsS -D - https://creapp.institucion.edu.ar/ruta-spa-profunda  
# Debe devolver text/html de la SPA, igual que /.
```

Complete también una prueba manual de login y de `/api/sadmin-creapp-panel/` (incluida página de login/admin); una API desconocida puede caer en el fallback SPA y contestar HTML 200. `/healthz` responde ok, pero no verifica conectividad con PostgreSQL.

4. Respaldos y Restauración

Crear una copia de seguridad (Backup)

Para respaldar la base de datos a un archivo comprimido mediante el comando oficial `pg_dump`:

```
umask 077  
pg_dump -h 127.0.0.1 -U creapp_usr -F c -f /var/backups/creapp_$(date +%F).dump creapp_db
```

Automatización (opcional): Puede programar la copia diaria a las 02:00 AM editando las tareas del usuario mediante `cron` (`crontab -e`):

```
0 2 * * * umask 077; pg_dump -h 127.0.0.1 -U creapp_usr -F c -f  
↪ /var/backups/creapp_$(date +%F).dump creapp_db
```

Restaurar una copia de seguridad (Restore)

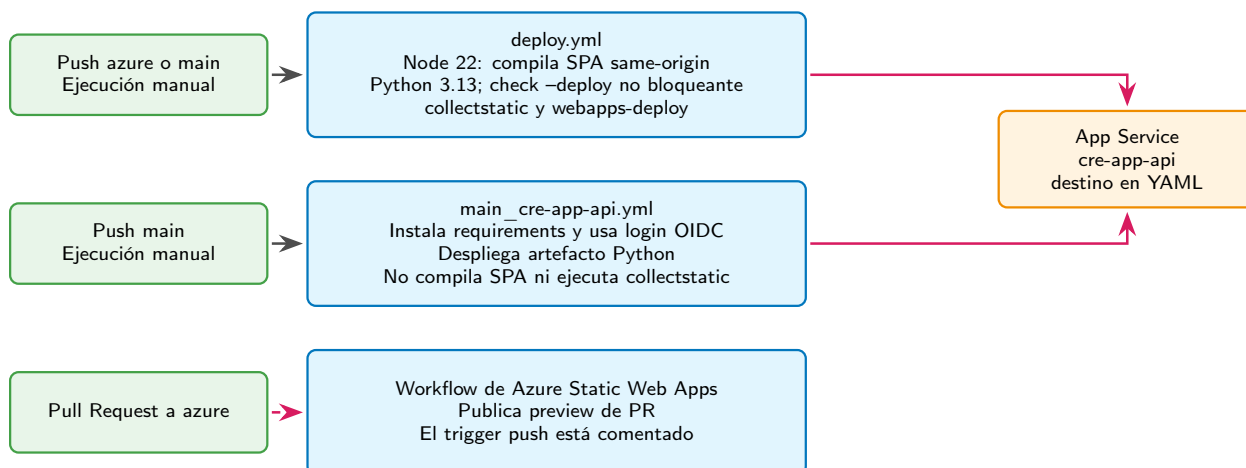
No restaure directamente sobre una base viva ni use `pg_restore --clean` como rollback. Haga backup del destino, restaure primero en una DB aislada, compruebe el código de salida, integridad, ownership y funcionamiento de la aplicación; acuerde una ventana y plan explícito antes de sustituir datos de producción. Un restore puede reemplazar o perder datos.

Ejemplo de prueba en una base aislada previamente creada:

```
sudo -u postgres createdb -O creapp_usr creapp_restore_test  
pg_restore -h 127.0.0.1 -U creapp_usr -d creapp_restore_test  
↪ /var/backups/creapp_YYYY-MM-DD.dump
```

5. Despliegue en Cloud Provider (Microsoft Azure)

Para entornos donde la institución requiera alojar CREApp en la nube de Microsoft Azure:



Un push a main puede iniciar ambos despliegues hacia el mismo App Service.
 Ninguno aplica migraciones; deploy.yml tampoco ejecuta tests y permite continuar si falla check -deploy.
 El commit activo, secretos, OIDC y estado del recurso se verifican en GitHub Actions y Azure Portal.

Figura 2. Flujos de despliegue en Azure y GitHub Actions

Paso 1: Aprovisionar Azure Database for PostgreSQL Flexible Server

1. Cree la instancia de [PostgreSQL Flexible Server](#) en Azure con autenticación por contraseña.
2. Configuración de red y cortafuegos:
 - **Firewall por IPs de salida:** Debe autorizar en el cortafuegos de PostgreSQL **todas** las direcciones listadas en las propiedades del App Service (tanto **Outbound IP addresses** como **Additional outbound IP addresses**), según las [reglas de red de Flexible Server](#). Las conexiones salientes de App Service toman una IP aleatoria del pool asignado en tiempo de ejecución; autorizar solo una causará fallos de conexión intermitentes.
 - **Seguridad de red:** No active la casilla *"Permitir acceso público desde cualquier servicio de Azure dentro de Azure"*, ya que no está restringida a su suscripción sino abierta a cualquier tenant de Azure. Emplee reglas de IP exactas o [Private Endpoint](#).
 - **Opción VNet / Private Endpoint:** Si la base de datos se ubica detrás de un Private Endpoint, el App Service requiere [VNet Integration](#) mediante una subred delegada dedicada.

Paso 2: Crear el recurso Azure App Service

Cree un **App Service (Linux)** seleccionando runtime **Python 3.13**, conforme a la [guía de configuración de Python en App Service](#) (Plan B1 o superior recomendado para producción).

Paso 3: Configurar variables en Application Settings

En la sección *Configuración* -> *Variables de entorno* del App Service en el portal de Azure, defina:

```

DEBUG=False
SECRET_KEY=clave_aleatoria_fuerte_de_50_caracteres
ALLOWED_HOSTS=<nombre-app>.azurewebsites.net
POSTGRES_DB=creapp_db
POSTGRES_USER=creapp_usr
POSTGRES_PASSWORD=contraseña_segura
POSTGRES_HOST=<servidor-db>.postgres.database.azure.com
  
```

```
POSTGRES_PORT=5432
CSRF_TRUSTED_ORIGINS=https://<nombre-app>.azurewebsites.net
CORS_ALLOWED_ORIGINS=https://<nombre-app>.azurewebsites.net
CSRF_COOKIE_SECURE=True
SESSION_COOKIE_SECURE=True
SECURE_SSL_REDIRECT=True
VITE_API_URL=/api
VITE_STATIC_BASE=/api/static/
```

Importante (Reinicio manual): Los cambios en las variables de entorno de App Service no recargan el proceso Django en caliente. Es **indispensable pulsar el botón Reiniciar (*Restart*) en el portal de Azure** para que las nuevas variables surtan efecto.

Paso 4: Configurar el comando de inicio (*Startup Command*)

En *Configuración general* del App Service, configure:

```
gunicorn --bind=0.0.0.0:8000 --timeout 600 --chdir backend config.wsgi:application
```

Paso 5: Despliegue automatizado con GitHub Actions

1. En el repositorio GitHub, guarde el perfil de publicación descargado de Azure en el secreto `AZURE_WEBAPP_PUBLISH_PROFILE`.
2. El workflow `.github/workflows/deploy.yml` compila la SPA con Node 22 (`VITE_API_URL=/api`, `VITE_STATIC_BASE=/api/static/`), instala requirements en Python 3.13, ejecuta `collectstatic` y publica el artefacto automáticamente al hacer push a `main` o `azure`, o mediante ejecución manual (*workflow_dispatch*). Vite vars son build-time, no runtime. Este workflow no ejecuta tests ni migraciones y `check --deploy` es no bloqueante (`|| true`); un run verde no prueba migraciones ni aplicación. `main` también activa `main_cre-app-api.yml` para la misma App Service y puede ocasionar un segundo despliegue. Revise cuál quedó activo y el artefacto final.

Paso 6: Verificación y arranque en frío (*Cold Start*)

El arranque inicial (*cold start*) de App Service puede demorar entre **30 y 90 segundos**. Configure sus sondas de monitoreo con un timeout no menor a 90 segundos antes de validar el endpoint `https://<nombre-app>.azurewebsites.net/healthz`.

6. Solución Rápida de Problemas Comunes

Las soluciones atienden a los códigos de estado y directivas de la especificación [RFC 9110 \(HTTP Semantics\)](#):

502 Bad Gateway

- **Causa habitual:** Gunicorn está detenido o Nginx no puede conectar a `127.0.0.1:8000`.
- **Solución inmediata:**
 1. Verifique el estado del servicio: `sudo systemctl status creapp`
 2. Compruebe los logs de error recientes: `sudo journalctl -u creapp -n 30`
 3. Compruebe que Gunicorn escucha en loopback con `sudo ss -ltnp | grep 127.0.0.1:8000` y que Nginx apunta al mismo host/puerto.

400 Bad Request

- **Causa habitual:** El nombre de dominio o IP solicitada no está registrado en `ALLOWED_HOSTS`.

- **Solución inmediata:** Edite `/etc/creapp/creapp.env` y agregue el dominio o IP a `ALLOWED_HOSTS`. Luego reinicie: `sudo systemctl restart creapp`.

403 Forbidden (CSRF verification failed)

- **Causa habitual:** El origen de la petición no coincide con `CSRF_TRUSTED_ORIGINS`.
- **Solución inmediata:** Asegúrese de acceder mediante `https://` y de que su dominio exacto esté configurado en `CSRF_TRUSTED_ORIGINS=https://tu-dominio` en `/etc/creapp/creapp.env`.

Error de conexión a PostgreSQL

- **Causa habitual:** Credenciales incorrectas, base de datos inexistente o servicio PostgreSQL inactivo.
- **Solución inmediata:**
 1. Verifique que PostgreSQL esté corriendo: `sudo systemctl status postgresql`
 2. Pruebe la conexión manual interactiva: `psql -h 127.0.0.1 -U creapp_usr -d creapp_db`

404 en imágenes / CSS / Estáticos

- **Causa habitual:** Falta el build Vite previo a `collectstatic`, no se generó `backend/staticfiles`, o Nginx alias no coincide con `STATIC_ROOT=/srv/creapp/backend/staticfiles` y `STATIC_URL=/api/static/`.
 - **Solución inmediata:** Construya primero la SPA con `npm ci` y `VITE_API_URL=/api` `VITE_STATIC_BASE=/api/static/` `npm run build` desde `/srv/creapp/frontend`; luego ejecute `collectstatic` con `EnvironmentFile` y `WorkingDirectory=/srv/creapp/backend` según los pasos anteriores. Confirme permisos de lectura de Nginx y `location ~ /api/static/`.
-

7. Documentación Oficial y Fuentes de Referencia

Para consultar en profundidad las especificaciones, parámetros y directivas oficiales:

- **Django:**
 - [Lista de comprobación para despliegue en producción](#)
 - [Gestión y publicación de archivos estáticos \(staticfiles\)](#)
 - [Referencia completa de configuración \(settings\)](#)
- **Gunicorn:**
 - [Guía de configuración y modelo de concurrencia](#)
 - [Despliegue e integración con Systemd](#)
- **Nginx:**
 - [Módulo de proxy inverso HTTP \(ngx_http_proxy_module\)](#)
 - [Configuración de servidores seguros HTTPS con SSL/TLS](#)
- **PostgreSQL:**
 - [Copias de seguridad lógicas \(pg_dump y pg_restore\)](#)
 - [Control de acceso y autenticación en pg_hba.conf](#)
 - [Gestión de roles y privilegios de usuario](#)
- **Microsoft Azure:**
 - [Aprovisionamiento y conectividad en Azure PostgreSQL Flexible Server](#)
 - [Configuración de aplicaciones Python en Azure App Service Linux](#)

- Integración de red virtual (VNet Integration) en App Service
- **Systemd y Linux:**
 - Especificación de unidades de servicio (systemd.service)
 - Control y filtrado de bitácoras del sistema con journalctl
- **Estándares y Protocolos Web:**
 - RFC 9110: HTTP Semantics (Códigos de estado y cabeceras de transporte)